Charter: Declarative Deliverable Contracts for Governing Long-Horizon Agent Runs

The `agent-charter` contributors
`github.com/zhenthebuilder/agent-charter`
This paper was produced and evaluated during an autonomous research run.

Abstract

Long-horizon agent tasks fail quietly: the agent finishes, sounds confident, and delivers artifacts that are missing sections, contain leftover scaffolding, or silently drop requirements stated hours earlier in the prompt. We argue that the deliverables of such runs should be governed the way software releases are governed—by a small, declarative, machine-checkable contract—and we present CHARTER, an open-source, dependency-free tool that turns a list of required artifacts and acceptance checks into (i) instructions for the agent, (ii) a deterministic gate with a CI-style exit code, and (iii) an audit report with file hashes. To quantify the value of gating generation on a deliverable contract, we run a paired A/B on IFEval, a public benchmark of 541 prompts with verifiable output constraints: single-shot generation versus the same model revised under a CHARTER gate compiled automatically from each prompt's constraints. The gate raises prompt-level strict accuracy from 87.1% to 93.9% (+6.8 points, McNemar $p = 7.8 \times 10^{-10}$) at a cost of 0.17 additional model calls per prompt on average, and the gain is concentrated almost entirely on constraints the contract can express—constraint types outside the contract's coverage are essentially unchanged. This both demonstrates the practical value of contract-gated generation and delineates its boundary: a gate can only fix what it can check. Code, contracts, raw logs, and scoring scripts are released for exact reproduction.

1 Introduction

Agents built on large language models are now routinely given tasks that run for hours: produce a research report with its underlying data, implement a feature with tests and documentation, analyze a dataset and write up the findings. The economics of these systems depend on the human *not* supervising every step. Yet the dominant failure mode of long-horizon runs is not a crash that pages someone—it is a confident "done" attached to deliverables that do not match what was asked: a missing section, a `TODO` left on page four, a data file that was never written, tests that never ran, instructions stated at hour zero that fell out of effective context by hour three.

Today the main control mechanism is post-hoc human review of everything the agent produced, which does not scale with agent throughput, or ad-hoc prompt engineering ("make sure the report has a limitations section"), which is not enforced by anything. LLM-as-a-judge pipelines (Zheng et al., 2023) help with fuzzy quality criteria but are stochastic, add cost to every iteration, and are hard to audit as a release gate. What is missing is the analogue of what made software delivery governable: a small, reviewable specification of "done" that a machine can check on every iteration, cheaply and deterministically.

We make that analogy concrete. CHARTER is a deliverable contract for agent runs: a declarative TOML/JSON file that lists the artifacts a run must produce and the acceptance checks each must pass—existence, size and length bounds, required sections, required or forbidden patterns, valid

JSON with required keys, absence of placeholder text, and arbitrary commands (test suites, linters, schema validators). The contract is used three ways by one tool: `charter explain` renders it as instructions injected into the agent's prompt; `charter check` evaluates the workspace and returns a CI-style exit code, with a structured failure report designed to be fed back to the model; `charter report` produces an audit trail (per-check verdicts, file hashes, timestamps) that a human can review instead of re-reading the transcript. CHARTER is a single stdlib-only Python file, integrates with CI systems, pre-commit, and agent frameworks (including a stop-hook mode that prevents an agent from declaring completion while required checks fail), and never calls a model in the verification path.

The empirical question is whether gating generation on such a contract actually changes outcomes, and at what cost. No public benchmark directly measures "deliverable governance for long-horizon runs," so we anchor the claim on the closest recognized proxy with verifiable acceptance criteria: IFEval (Zhou et al., 2023), 541 prompts each annotated with programmatically checkable constraints (word counts, required keywords, formatting, etc.). We compile each prompt's constraints into a CHARTER contract using only the tool's generic check primitives, and compare single-shot generation against a gate-and-revise loop in a paired design. Three results stand out. First, the gate recovers a large fraction of the compliance that a capable model misses single-shot (+6.8 points prompt-level strict accuracy). Second, the recovery is concentrated on constraints the contract can express: instruction types our compiler covers exactly improve by 4.2 points, while types it cannot express move by only 0.0 points—the gate's value is bounded by its coverage, which we measure and report. Third, the cost is modest and predictable (0.17 extra model calls per prompt on average, incurred only when a check fails).

We do not claim that contract gating improves a model's underlying capabilities, nor that IFEval is a long-horizon benchmark. The claim is narrower and, we believe, more useful: when acceptance criteria can be written down as deterministic checks—which covers a surprisingly large share of what "done" means for real deliverables—putting those checks in the loop converts silent non-compliance into either compliance or an explicit, auditable failure.

Our contributions are: (1) a problem framing—deliverable contracts—for governing the outputs of long-horizon agent runs, separating what is verifiable mechanically from what requires human or model judgment; (2) CHARTER, a production-quality, dependency-free implementation designed for adoption inside existing CI and agent-loop infrastructure; (3) a paired evaluation on a public benchmark quantifying both the benefit and the boundary (coverage) of contract-gated generation, with all logs, contracts, and scoring code released.

2 Related Work

Verifiable instruction following. IFEval (Zhou et al., 2023) established the practice of evaluating LLMs on constraints that can be checked by a program rather than a judge, and is part of standard open evaluation suites. We use it not as a capability benchmark but as a source of ground-truth acceptance criteria against which a governance mechanism can be measured with no judge in the loop.

Self-correction and refinement. Self-Refine (Madaan et al., 2023) and related work show that models can improve their own outputs given feedback; Huang et al. (2024) show that *intrinsic* self-correction—without external signals—often fails or degrades performance, and Pan et al. (2024) survey the space of correction strategies. Our results are consistent with this literature: the gains in our treatment arm come from cheap, *external*, deterministic verification compiled from the task specification, not from asking the model to critique itself. CHARTER's contribution is not a new correction algorithm but the contract abstraction and tooling that make external verification deployable in real agent workflows.

Constrained and guided generation. Grammar- and automaton-constrained decoding (Willard and Louf, 2023) guarantees syntactic properties (e.g., JSON well-formedness) at decoding time, but requires logit-level access and cannot express cross-artifact, filesystem-level criteria ("the report cites at least five sources and the tests pass"). Contract gating operates after generation, treats the generator as a black box, and composes with constrained decoding where available.

Evaluating agents. Agent benchmarks such as SWE-bench (Jimenez et al., 2024), GAIA (Mialon et al., 2023), and AgentBench (Liu et al., 2023) score end-task success of agent systems. They answer "how capable is this agent?"; we address the complementary, deployment-time question "did *this run* deliver what *I* specified, and can I verify that without reading everything?". LLM-as-a-judge evaluation (Zheng et al., 2023) is the prevailing answer for criteria that resist mechanical checks; CHARTER deliberately handles the deterministic complement and can invoke judges through its `command` check where teams choose to.

Design by contract and CI. The contract framing borrows directly from design-by-contract in software engineering (Meyer, 1992) and from the practice of gating merges and releases on automated checks. Our observation is that agent deliverables currently lack the equivalent layer, even though the artifacts they produce (files in a workspace) are exactly what such tooling is good at checking.

3 Charter: deliverable contracts for agent runs

Contract model. A charter is a TOML or JSON document with a list of *deliverables*, each with an identifier, an optional path or glob, a required/optional flag, and a list of *checks*. Checks are drawn from a small set of primitives chosen to be deterministic, dependency-free, and fast: `exists`, `not_exists`, `file_count`, `min_bytes`, `max_bytes`, `contains` / `not_contains` (literal or regex, with occurrence bounds), `word_count`, `line_count`, `json_valid` (with required dotted keys), `markdown_sections`, `no_placeholder` (TODO/FIXME/lorem-ipsum style scaffolding), and `command`, which delegates to any executable check the team already trusts (test suites, linters, schema validators) and subsumes domain-specific verification. Checks carry a severity (`error` or `warn`); a run fails if any required deliverable has a failing error-severity check.

Three uses of one artifact. The same contract is (i) *rendered for the agent*: `charter explain` produces a human- and agent-readable statement of what must exist and how it will be judged, suitable for inclusion in the system or task prompt; (ii) *enforced*: `charter check` evaluates the workspace and exits 0 or 1, prints a structured failure report (also available as JSON), and has a hook mode in which a failing contract blocks an agent's attempt to finish and feeds the failure list back into the loop; and (iii) *audited*: `charter report` emits per-check verdicts together with SHA-256 hashes, sizes, and timestamps of the artifacts, so a reviewer can trust the verdict without re-deriving it. Because verification is deterministic and local, the gate can run on every agent iteration at negligible cost, and the same contract yields the same verdict for the agent, the CI system, and the human.

What contracts deliberately do not do. Charter does not judge qualities that require taste or domain expertise (is the analysis insightful?), does not verify factual grounding, and does not attempt to be a sandbox or permission system. It narrows the surface a human must review, and makes explicit—in a reviewable file rather than in prompt history—which acceptance criteria are mechanically enforced.

4 Compiling a public benchmark's constraints into contracts

To measure the effect of contract gating we need tasks whose acceptance criteria are known and independently scoreable. IFEval provides 541 prompts with 834 verifiable instruction instances across 25 types. A small compiler (`compile_contract.py`, 200 lines) maps each instruction onto CHARTER's generic primitives and labels every instance with a *coverage class*:

- **exact** (451 instances, 54.1%): the compiled check is logically equivalent to the official checker, e.g. keyword presence/frequency, forbidden words, no-comma, postscript, title markers, end-phrase, quotation wrapping, placeholder counts.
- **approximate** (288 instances, 34.5%): the check captures the constraint with minor semantic differences, e.g. whitespace word counts versus the official tokenizer, bullet/paragraph counting via regular expressions, all-caps/all-lowercase via character classes, JSON validity without markdown-fence stripping.

Table 1: Paired comparison on IFEval (541 prompts), official scorer. The treatment differs from control only where a compiled CHARTER check failed and triggered revision.

Metric	Control	+ CHARTER gate	Δ
Prompt-level strict accuracy (%)	87.1	93.9	+6.8
Prompt-level loose accuracy (%)	90.4	95.4	+5.0
Instruction-level strict accuracy (%)	91.0	96.0	—

Table 2: Instruction-level strict accuracy by CHARTER coverage class. The gate improves what it can check; constraints it cannot express are essentially unchanged.

Coverage class	Instances	Control (%)	+ gate (%)	Δ
Exact	451	94.2	98.4	+4.2
Approximate	288	89.6	97.6	+8.0
Uncovered	95	80.0	80.0	+0.0
All	834	91.0	96.0	+5.0

- **uncovered** (95 instances, 11.4%): not expressible with the generic primitives—response language detection, sentence counting, and first-word-of-nth-paragraph constraints.

The compiler is part of the released harness. Coverage is reported per instance in the run logs, and Section 6 breaks results down by class. We emphasize the design choice this reflects: in deployment, the contract author decides what is worth checking mechanically; here the benchmark's own constraint annotations play that role, which makes the overlap between gate and scorer intentional rather than confounding—the experiment measures how much of the specified-and-checkable requirements a strong model silently misses, and how much of that a contract gate recovers.

5 Experimental setup

Arms. **Control**: one model call per prompt; the response is used as-is. **Treatment**: the treatment starts from the *same* control response (exact pairing, isolating the effect of the gate from sampling noise); the compiled contract is evaluated by the real `charter check` CLI on the response written to a file; if any check fails, the structured failure report is appended to a revision request and the model produces a new response, for at most two revisions; the gate keeps the attempt with the fewest failing checks. Prompts whose initial response already passes the contract incur no additional calls and are byte-identical across arms.

Model and harness. Both arms use `claude-haiku-4-5` (Anthropic) called through the Claude Code CLI in non-interactive print mode with a minimal "helpful assistant" system prompt and no tool access; the identical stack is used for both arms, and all calls are logged verbatim. We chose a small frontier model deliberately: it is representative of the cost tier at which long-horizon loops actually run, and it leaves measurable headroom on IFEval.

Scoring and statistics. All accuracy numbers are computed by the official, unmodified `instruction_following_eval` library released with IFEval (vendored in our repository), in both strict and loose modes, at the prompt level (all instructions of a prompt satisfied) and the instruction level. Because arms are paired by prompt, we report McNemar's exact test on discordant pairs and a nonparametric bootstrap (10,000 resamples) for the 95% confidence interval of the accuracy difference. The full pipeline—harness, contracts, raw responses for every attempt, scoring and analysis scripts—is committed alongside this paper, and every number below is generated programmatically from those artifacts.

6 Results

Main effect. Table 1 summarizes the paired comparison. Single-shot generation with a capable small model satisfies all of a prompt's verifiable constraints 87.1% of the time under the official strict scorer; under the CHARTER gate this rises to 93.9% (+6.8 points, 95% bootstrap CI [+4.6, +9.1], McNemar exact $p = 7.8 \times 10^{-10}$). The discordant pairs are heavily one-sided: 39 prompts are satisfied only under the gate versus 2 only under control. Loose-mode accuracy moves similarly (+5.0 points), indicating the gains are not artifacts of strict-mode formatting technicalities. Both regressions (prompts satisfied only under control) occurred on approximately-covered constraints, where a revision satisfied the compiled approximation but violated the official checker's stricter semantics—a concrete cost of approximate coverage that Section 7 returns to.

The gain follows coverage. Table 2 breaks instruction-level strict accuracy down by the compiler's coverage class. Exactly-covered constraint types improve by 4.2 points and approximately-covered types by 8.0 points, while uncovered types move by only 0.0 points—within noise of no change, as expected since the gate cannot observe them. This is the central honest finding: contract gating is not a general capability amplifier; it converts *specified and checkable* requirements from "usually followed" to "almost always followed," and does nothing for requirements left outside the contract.

Gate dynamics and cost. Of 541 prompts, 61 failed at least one compiled check on the initial response; after at most two gated revisions, 22 still failed at least one check. The gate selected a revised response for 61 prompts; for all others the original response was already contract-clean and the arms are identical. The loop cost 90 additional model calls in total (0.17 per prompt on average, concentrated on the 61 prompts that needed revision), and mean response length was essentially unchanged (1537 versus 1546 characters), i.e. the gate did not induce padding. There is no verification-time model cost: checks are regular expressions and file inspections.

Qualitative observation from dogfooding. The research run that produced this paper was itself governed by a charter covering its required deliverables. The gate caught, among other things, marketing copy that tripped a placeholder-text check because it *described* placeholder text—a false positive that was resolved by narrowing the contract, illustrating the intended workflow: acceptance criteria are tuned in a reviewable file, not in prompt history.

7 Limitations

Benchmark proxy. IFEval is single-turn instruction following, not a long-horizon agent benchmark; we use it because it is public, recognized, and has verifiable acceptance criteria, but the transfer of effect sizes to multi-hour, multi-artifact runs is an extrapolation. Building a public benchmark of contract-governed long-horizon deliverables is future work. **Gate–scorer overlap.** The treatment optimizes checks compiled from the same constraint annotations the scorer uses; results quantify recovery of specified-and-checkable requirements, not generalization to unspecified ones (Table 2 makes the boundary explicit). **Coverage ceiling.** 11.4% of constraint instances were not expressible with generic primitives, and approximate checks can both miss violations and raise false alarms; richer primitives (or `command`-delegated domain validators) shift but do not remove this boundary. **Single model, single revision policy.** We evaluate one model at one cost tier with at most two revisions; gains may differ for stronger models (less headroom) or weaker ones (revisions may fail more often). **Contracts can be gamed.** A model optimizing against a literal check can satisfy it in unintended ways (e.g., padding to reach a word count); deterministic gates should be combined with human or model review for adversarial settings.

8 Conclusion

Treating the deliverables of long-horizon agent runs as the subject of a small, declarative, machine-checkable contract is both practically deployable today and measurably effective at eliminating the silent non-compliance that currently forces humans to re-read everything an agent produces. The boundary of the approach is exactly its coverage—a property we can measure, report, and expand

deliberately—which we argue is the right shape for a governance mechanism: it is auditable about what it does and does not guarantee. CHARTER, the benchmark harness, raw logs, and this paper's numbers are released for reproduction and adoption.

References

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.01798.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating LLMs as agents. *arXiv preprint arXiv:2308.03688*, 2023.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2303.17651.

Bertrand Meyer. Applying "design by contract". *IEEE Computer*, 25(10):40–51, 1992.

Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. GAIA: a benchmark for general AI assistants. *arXiv preprint arXiv:2311.12983*, 2023.

Liangming Pan, Michael Saxon, Wenda Xu, Deepak Nathani, Xinyi Wang, and William Yang Wang. Automatically correcting large language models: Surveying the landscape of diverse automated correction strategies. *Transactions of the Association for Computational Linguistics*, 12:484–506, 2024. arXiv:2308.03188.

Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*, 2023.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2306.05685.

Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models. *arXiv preprint arXiv:2311.07911*, 2023.